

Presentation Script — EPN Survival Analysis

Presentation Script — English

Modernization and Integration of a New Graph-Based Deep Learning Method for Survival Analysis

Yosuke Kimoto & Sena Fukabe — ECN 2026

SLIDE 01 — Title

Good morning, everyone. My name is Yosuke Kimoto, and together with Sena Fukabe, we will present our project. The topic is the integration of a graph-based deep learning method called the Error Propagation Network into an existing survival analysis pipeline. We have about 20 minutes, so let's get started.

SLIDE 02 — Agenda

Here is our plan for today. We will start with background on survival analysis. Then we explain our project objective, followed by an overview of the three models we worked with. After that, we go into the implementation details. Then we show our experimental results, and finish with discussion and future work.

SLIDE 03 — Background: Survival Analysis

First, what is survival analysis? The goal is to predict the survival function $S(t|x)$ — the probability that a patient survives beyond a given time t . The main challenge is censored data: patients who left the study before experiencing the event. Standard machine learning cannot handle this directly.

We use two metrics. The C-index measures ranking accuracy. 0.5 means random, and 1.0 is perfect. The IBS measures the mean squared prediction error. Lower is better.

Our dataset is METABRIC — 1,904 breast cancer patients with 9 clinical features. We use 5-fold cross-validation for all experiments.

SLIDE 04 — Project Objective

Our main objective was to integrate EPN into PyTorch Lightning pipeline.

We followed five steps. First, verify the existing SurvivalDGM pipeline. Second, wrap the MLP as a Lightning module. Third, build a patient similarity graph using PyTorch Geometric. Fourth, reimplement EPN as a tensor-only Lightning module. Fifth, add Hydra for configuration management.

SLIDE 05 — Three Models at a Glance

We worked with three models.

SurvivalDGM is the existing baseline from Valentin. It uses a probabilistic graph and achieves a mean C-index of 0.6043.

Our MLP is Stage 1 of our pipeline. It is a simple feedforward network: $9 \rightarrow 64 \rightarrow 64 \rightarrow 1$, trained with CoxPH loss. It achieves the best result: 0.6495.

EPN is our main contribution — Stage 2 of the pipeline. It is a graph attention network that corrects MLP predictions using neighbour errors. It achieves 0.6157.

SLIDE 06 — Stage 1: MLP Architecture

The MLP takes 9 clinical features as input. It passes through two hidden layers of size 64 with ReLU activation. The output is a risk score, which the PyCox library converts into survival curves.

The final output is a matrix of size N by T, where T is about 1,391 time points. This matrix is the input to Stage 2.

We trained for 110 epochs with Adam, learning rate 0.001.

SLIDE 07 — Stage 2: Error Propagation Network

The core idea of EPN is simple. If the MLP makes similar mistakes for patients similar to patient A, we can use those mistakes to correct A's prediction.

The process has five steps. First, find $k=10$ nearest neighbours in the patient graph. Second, compute the MLP prediction error for each neighbour. Third, compute attention weights using learned projections. Fourth, compute a weighted sum of neighbour errors as the correction. Fifth, add the correction to the MLP prediction.

The training loss is CoxCC — a case-control approximation of CoxPH.

SLIDE 08 — Repository Structure

Here is the repository structure. Gray files are the original code that already exist. Blue files with a star are what we added.

We added `graph_builder.py` and `epn_datamodule.py` in the data layer. We added `mlp_module.py` and `epn_module.py` as Lightning modules. We added scripts for training, running the full pipeline, and visualization.

SLIDE 09 — Two-Stage Pipeline

The pipeline runs in two stages.

In Stage 1, we train the MLP for 110 epochs. After training, we call `mlp.freeze()` to lock the weights permanently.

In Stage 2, `EPNDataModule` precomputes MLP survival curves for all patients and builds the k-NN graphs. Then we train the EPN for 50 epochs.

We repeat this for all 5 folds.

SLIDE 10 — Patient Similarity Graph Construction

We use PyTorch Geometric to build the graph. We compute cosine similarity between patients based on their 9 features. Each patient gets k=10 nearest neighbours.

We build two graphs. The training graph connects training patients only. The validation graph connects validation patients to training patients only. This prevents data leakage.

SLIDE 11 — Engineering Challenge 1: DataFrame Removal

Our first challenge was the DataFrame dependency. The original code passed a pandas DataFrame to `forward()`. This is incompatible with Lightning's tensor pipeline.

We refactored the interface to use only tensors: features, survival predictions, labels, and neighbour indices.

SLIDE 12 — Engineering Challenge 2: Memory Optimization

Our second challenge was memory. The naïve approach requires about 12.9 gigabytes — this caused an out-of-memory crash.

We applied two optimizations. First, mini-batching: process 256 patients per step instead of all 1,523. Second, graph-constrained attention: attend only to k=10 neighbours, not all patients.

Together, these reduce memory to about 14 megabytes — a 26-times reduction.

SLIDE 13 — Hydra Configuration Management

We used Hydra to manage hyperparameters through YAML files. All settings for the MLP, EPN, and graph are in one config file.

The key benefit is that we can override any parameter from the command line. No source code changes needed. This makes experiments reproducible and easy to compare.

SLIDE 14 — Patient Graph Visualization

This figure shows the patient similarity graph projected into 2D using PCA. Blue points are censored patients, orange points experienced the event.

The two groups are interleaved — there is no clear separation in the feature space. This explains the modest C-index values across all models.

SLIDE 15 — Survival Curve Comparison

This figure shows survival curves for four patients. Green is the MLP baseline, orange dashed is the EPN-corrected prediction.

For event patients in the top row, EPN shifts the curve downward after the event time. The error signal from similar neighbours is working.

For censored patients in the bottom row, the correction is smaller. Censored observations provide a weaker error signal.

SLIDE 16 — Quantitative Results

Here are the numbers.

The MLP achieves a mean C-index of 0.6487, and EPN achieves 0.6159.

In the three-model comparison: MLP is the best at 0.6495. EPN achieves 0.6157 — better than SurvivalDGM's 0.6043.

So EPN surpasses the existing baseline, but does not yet beat the MLP.

SLIDE 17 — Discussion

Why does EPN not surpass the MLP?

METABRIC is a relatively small dataset — about 1,900 patients with 9 features. The MLP with CoxPH loss is already a strong baseline. Graph-based methods tend to work better on larger datasets.

There are four specific reasons. One: hyperparameters were not tuned. Two: feature similarity does not always imply error similarity. Three: 1,523 training patients may be too few for effective

graph correction. Four: MLP and EPN use different loss functions, making it hard to isolate the graph's contribution.

SLIDE 18 — Future Improvements

We propose five directions.

Tune the graph size k over $\{5, 10, 20, 50\}$. Train EPN for more epochs. Use CoxPH for EPN for a fair comparison. Explore learned graph construction. Test on SUPPORT — about 9,000 patients — where graph methods should be more effective.

SLIDE 19 — Conclusion

Our engineering achievements: We reimplemented EPN as a tensor-only Lightning module. We reduced memory from 12.9 GB to 14 MB — 26 times smaller. We integrated Hydra for reproducible experiments.

Our results: MLP: C-index = 0.6495 — best. EPN: C-index = 0.6157 — beats the SurvivalDGM baseline.

EPN surpasses the baseline. The most promising next step is tuning k and training duration.

Thank you. We are happy to take questions.

SLIDE 20 — References

(Display only — no script needed)

BACKUP SLIDES — Q&A

B1 — CoxPH vs CoxCC

CoxPH uses all patients who have not yet failed as the risk set at each event time. It handles censored data naturally, but is expensive for large datasets.

CoxCC replaces the full risk set with a small random sample of controls. This reduces computation while approximating the CoxPH gradient. It is the standard approach for large-scale survival deep learning.

B2 — Data Leakage Prevention

If validation patients could attend to each other's errors, information from the validation set would leak into the model.

We prevent this with two separate graphs. Training graph: training patients to training patients only. Validation graph: validation patients to training patients only. Validation patients cannot see each other.

B3 — SurvivalDGM Evaluation

SurvivalDGM samples each edge from a Bernoulli distribution. Every forward pass produces a different graph, so a single-pass C-index is unstable. We average over 100 forward passes per fold. EPN uses a fixed k-NN graph — deterministic, no repeated sampling needed.

B4 — Hydra Config Tree

The config tree is composable. You can mix and match config files and override any value from the command line. No source code changes are needed for new experiments.

Q&A — Anticipated Questions

Q1 — Why did you use 5-fold cross-validation?

English: METABRIC has only 1,904 patients — a relatively small dataset for deep learning. With a single train/test split, the result would depend heavily on which patients happen to fall in each set. 5-fold CV uses every patient for both training and testing across five rotations, giving a more reliable and less biased performance estimate. It is the standard protocol for survival analysis on small clinical datasets.

日本語: METABRICは約1,900人と、深層学習としては小規模なデータセットです。1回だけの分割では、結果がたまたまの分割に大きく左右されてしまいます。5分割交差検証では、全患者を5回にわたってトレーニングとテストの両方に使えるため、より信頼性の高い性能評価が得られます。臨床データの生存分析では標準的な手法です。

Q2 — Why these 5 implementation steps? What does each mean?

English: The five steps mirror the dependency chain of the system. Step 1 (verify SurvivalDGM) establishes the baseline we compare against. Step 2 (MLP as Lightning module) converts Stage 1 into a standardized, reusable component. Step 3 (patient graph with PyG) builds the graph structure that EPN needs as input. Step 4 (EPN as Lightning module) implements the graph-attention correction in the same framework. Step 5 (Hydra) ties everything together under reproducible configuration. Skipping any step would break the downstream pipeline.

日本語: 5つのステップは、システムの依存関係の順序を反映しています。ステップ1 (SurvivalDGMの検証) は比較対象のベースラインを確立します。ステップ2 (MLPのLightningモジュール化) はステージ1を標準化された再利用可能なコンポーネントに変換します。ステップ3 (患者グラフ構築) はEPNが必要とするグラフ構造を作ります。ステップ4 (EPNのLightningモジュール化) はグラフアテ

ンションによる補正を同じフレームワークで実装します。ステップ5 (Hydra) で全体を再現可能な設定管理のもとにまとめます。どのステップを省いても後続のパイプラインが壊れます。

Q3 — Why compare these 3 models? What does each represent?

English: Each model represents a different design philosophy for survival analysis. SurvivalDGM is the existing baseline — a stochastic graph model with learned edge probabilities. MLP is our Stage 1 — a strong, simple feedforward baseline trained with the gold-standard CoxPH loss. EPN is our main contribution — a graph-attention network that corrects the MLP using neighbour error signals. Comparing all three lets us answer two questions: does our MLP improve on SurvivalDGM, and does adding graph-based correction improve on the MLP alone?

日本語: 各モデルは生存分析の異なる設計哲学を代表しています。SurvivalDGMは既存のベースライン — 確率的グラフモデルです。MLPはステージ1として、CoxPH損失で訓練されたシンプルで強力なベースラインです。EPNが今回の主な貢献 — 近傍の誤差シグナルを使ってMLPの予測を補正するグラフアテンションネットワークです。3モデルを比較することで、「MLPはSurvivalDGMより良いか」「グラフ補正はMLPよりさらに改善するか」の2つの問いに答えられます。

Q4 — Is this an integration of MLP and EPN into the existing pipeline? What happened to SurvivalDGM?

English: Yes, we added MLP and EPN as new modules inside the existing Valentin pipeline. SurvivalDGM remains unchanged — its code lives in `dgm_model.py` and we evaluated it as-is to obtain the baseline C-index of 0.6043. We did not modify or replace it; we simply built a new two-stage pipeline alongside it for comparison.

日本語: はい、既存のValentinパイプラインの中にMLPとEPNを新しいモジュールとして追加しました。SurvivalDGMはそのまま残っており、`dgm_model.py`のコードは変更していません。ベースラインのC-index 0.6043を得るために、そのまま評価しました。修正や置き換えは行わず、比較用として新しい2段階パイプラインを別途構築しました。

Q5 — What are the 9 input features of the MLP?

English: The 9 features come from the standard pycox METABRIC dataset. Five are continuous and are standardized: MKI67, EGFR, PGR, ERBB2, and age at diagnosis. Four are binary and are passed through unchanged: hormone treatment, radiotherapy, chemotherapy, and ER-positive status. These are the canonical clinical features used in most METABRIC survival analysis benchmarks.

日本語: 9つの特徴量はpycoxのMETABRICデータセットの標準的なものです。連続値の5つは標準化されます: MKI67、EGFR、PGR、ERBB2、診断時年齢。バイナリ値の4つはそのまま使用: ホルモン療法、放射線療法、化学療法、ER陽性。これらはMETABRICの生存分析ベンチマークで広く使われている標準的な臨床特徴量です。

Q6 — Why CoxPH for MLP and CoxCC for EPN? What do they each mean?

English: CoxPH (Cox partial likelihood) uses the full risk set — all patients still at risk — at each event time. It is the exact formulation for the Cox model and handles censored data naturally. For the MLP, which processes all training data at once, this is computationally feasible.

CoxCC (Cox case-control) approximates CoxPH by replacing the full risk set with a small random sample of controls. EPN trains with mini-batches of 256 patients, so computing the full risk set at every step would be prohibitively expensive. CoxCC is the standard solution for this setting in deep survival analysis.

日本語: CoxPH (Coxの部分尤度) は各イベント時点でリスク集合 (まだイベントを経験していない全患者) を使います。Cox比例ハザードモデルの正確な定式化であり、打ち切りデータを自然に扱えます。MLPは全訓練データを一度に処理するため、計算コストの面で実行可能です。

CoxCC (Coxのケースコントロール近似) は、リスク集合の代わりにランダムサンプルした少数のコントロールを使います。EPNは256人のミニバッチで訓練するため、各ステップで完全なリスク集合を計算するのはコストが高すぎます。CoxCCは深層生存分析のこの設定における標準的な解決策です。

Q7 — Which file do you actually run? Is the goal to create `run_epn_v2.py`?

English: The main entry point is `run_epn_v2.py`. Running `python -m survival_analysis.experiments.run_epn_v2` from the Vanlentin's pipeline directory executes the full two-stage pipeline for all 5 folds. Yes, making this script work correctly — with clean tensor interfaces, memory-efficient training, and Hydra configuration — was the central engineering goal of the project.

日本語: メインのエントリーポイントは`run_epn_v2.py`です。Vanlentin's pipelineディレクトリから`python -m survival_analysis.experiments.run_epn_v2`を実行すると、5分割全ての2段階パイプラインが実行されます。はい、このスクリプトをきれいなテンソルインターフェース、メモリ効率の良い訓練、Hydra設定管理で正しく動作させることが、このプロジェクトの中心的な工学的目標でした。

Q8 — What is the role of each new file? Which are essential vs. unnecessary?

English: Every file added serves a distinct purpose. `graph_builder.py` constructs the k-NN patient similarity graph — essential for EPN. `epn_datamodule.py` precomputes MLP curves and builds graphs before EPN training — the essential bridge between Stage 1 and Stage 2. `mlp_module.py` wraps the MLP as a Lightning module — essential for standardized training. `epn_module.py` implements the graph-attention correction — the core contribution. `run_mlp.py` trains the MLP for all 5 splits — used to get MLP-only results. `run_epn_v2.py` runs the full pipeline — the main entry point. `visualize.py` generates the graph and survival curve

plots — used for the slides, not required for results. All files except `visualize.py` are essential to reproduce the experimental results.

日本語：追加した各ファイルはそれぞれ明確な役割を担っています。 `graph_builder.py`：k-NN患者類似グラフを構築 — EPNに必須。 `epn_datamodule.py`：EPN訓練前にMLPの生存曲線を事前計算し、グラフを構築 — ステージ1と2をつなぐ橋渡し役として必須。 `mlp_module.py`：MLPをLightningモジュールとしてラップ — 標準化された訓練に必須。 `epn_module.py`：グラフアテンション補正を実装 — コアの貢献。 `run_mlp.py`：5分割全てでMLPを訓練 — MLP単体の結果に使用。 `run_epn_v2.py`：パイプライン全体を実行 — メインのエントリーポイント。 `visualize.py`：グラフと生存曲線のプロットを生成 — スライド用で、結果再現には不要。 `visualize.py`以外は全て実験結果の再現に必須です。

Q9 — Why 110 epochs for MLP and 50 for EPN?

English: Both values were chosen empirically based on observed convergence. The MLP is trained from random initialization and needs more epochs to learn stable survival representations from 9 features. EPN starts from a frozen, already-trained MLP and only needs to learn the correction signal, so convergence is faster. These numbers were not formally tuned — systematic hyperparameter search is listed as a future improvement.

日本語：両方の値は観測された収束に基づいて経験的に選択しました。MLPはランダム初期化から始まり、9つの特徴量から安定した生存表現を学ぶためにより多くのエポックが必要です。EPNはすでに訓練された凍結済みのMLPから始まり、補正シグナルだけを学べばよいため、収束が速いです。これらの値は正式にチューニングしたわけではなく、体系的なハイパーパラメータ探索は今後の改善課題として挙げています。

Q10 — How do the files interact during 2-stage training? Which file runs the training?

English: `run_epn_v2.py` orchestrates everything. For each fold, it first calls the MLP training loop using `mlp_module.py` with a PyTorch Lightning Trainer. After training, it calls `mlp.freeze()` to lock the MLP weights permanently. Then `EPNDataModule` (from `epn_datamodule.py`) precomputes the MLP survival curves for all patients and calls `graph_builder.py` to build the k-NN graphs. Finally, a second Lightning Trainer trains `epn_module.py` using those precomputed curves and graphs. This repeats for all 5 folds.

日本語：`run_epn_v2.py`が全体を統括します。各フォールドで、まずPyTorch Lightning Trainerを使って`mlp_module.py`によるMLPの訓練ループを実行します。訓練後、`mlp.freeze()`を呼んでMLPの重みを永久に固定します。次に`EPNDataModule` (`epn_datamodule.py`) が全患者のMLP生存曲線を事前計算し、`graph_builder.py`を呼んでk-NNグラフを構築します。最後に、2つ目のLightning Trainerがその事前計算した曲線とグラフを使って`epn_module.py`を訓練します。これを全5分割で繰り返します。

Q11 — Why PyTorch Lightning? What is it?

English: PyTorch Lightning is a high-level framework that wraps raw PyTorch. It standardizes the training loop — handling epoch iteration, optimizer steps, gradient zeroing, validation, and device placement — so you only write the model logic. We adopted it because the existing SurvivalDGM pipeline already uses Lightning. Using the same framework for MLP and EPN ensures consistency and makes it easy to swap trainers or add callbacks like early stopping.

日本語: PyTorch Lightningは生のPyTorchをラップした高レベルフレームワークです。エポックのイテレーション、オプティマイザのステップ、勾配のリセット、検証、デバイス配置など、訓練ループを標準化してくれるので、モデルのロジックだけを記述すればよくなります。既存のSurvivalDGMパイプラインがすでにLightningを使っているため採用しました。同じフレームワークをMLPとEPNに使うことで一貫性が保たれ、トレーナーの変更や早期終了などのコールバック追加も容易になります。

Q12 — When and where is the patient similarity graph used? Why build it?

English: The graph is built in `EPNDataModule.setup()`, after the MLP is frozen, at the start of Stage 2. It is used inside `epn_module.forward()` to define which neighbours each patient attends to. Without the graph, EPN has no structure — it cannot know which patients are similar and cannot aggregate their error signals. The k-NN graph is the mechanism that gives EPN its ability to correct predictions using local neighbourhood information.

日本語: グラフはMLPが凍結された後、ステージ2の開始時に`EPNDataModule.setup()`内で構築されます。`epn_module.forward()`の中で、各患者がどの近傍に注目するかを定義するために使われます。グラフなしではEPNに構造がなく、どの患者が似ているかを知ることができず、誤差シグナルを集約することもできません。k-NNグラフは、EPNがローカルな近傍情報を使って予測を補正する仕組みの核心です。

Q13 — Why PyTorch Geometric?

English: PyTorch Geometric (PyG) is the standard library for graph neural networks in PyTorch. It provides efficient sparse operations for message passing, the `Data` class for representing graphs as tensors, and graph-aware data loaders. Building this from scratch in raw PyTorch would require reimplementing graph convolution kernels, sparse attention, and batching logic — all well-tested features already in PyG. Using PyG also ensures compatibility with future graph architectures if we extend the project.

日本語: PyTorch Geometric (PyG) はPyTorchでグラフニューラルネットワークを構築するための標準ライブラリです。メッセージパッシングのための効率的なスパース演算、グラフをテンソルで表現する`Data`クラス、グラフ対応のデータローダーを提供します。生のPyTorchで一から作るにはグラフ畳み込みカーネル、スパースアテンション、バッチング処理をすべて再実装する必要があります。PyGを使うことで、プロジェクトを拡張した場合の将来のグラフアーキテクチャとの互換性も確保できます。

Q14 — What is cosine similarity? Why use it?

English: Cosine similarity measures the angle between two feature vectors, regardless of their magnitude. It equals 1 when vectors point in the same direction and 0 when they are orthogonal. We use cosine similarity because it is scale-invariant — a patient with uniformly large feature values is not automatically considered similar to another. What matters is the relative pattern across features, not the absolute scale. This is more appropriate than Euclidean distance for mixed-scale clinical features.

日本語: コサイン類似度は2つの特徴ベクトルの大きさに関係なく、向きの一致度（角度）を測ります。同じ方向を向いているとき1、直交しているとき0になります。スケール不変性があるため採用しました — 全特徴量の絶対値が大きい患者が自動的に「似ている」と判定されることがありません。重要なのは特徴量間の相対的なパターンであり、絶対的なスケールではありません。スケールが混在する臨床特徴量にはユークリッド距離より適しています。

Q15 — Why k=10 for the patient graph?

English: k=10 is the default used in the original `surv_epn` paper, so we adopted it as a principled starting point. It provides a balance: enough neighbours to generate a meaningful error correction signal, without connecting patients to dissimilar ones far away in the feature space. Too small a k means the correction signal is noisy; too large a k includes irrelevant neighbours. Tuning k over a range such as {5, 10, 20, 50} is explicitly listed as a future improvement in our discussion.

日本語: k=10はオリジナルの`surv_epn`論文で使われているデフォルト値であり、根拠のある出発点として採用しました。これは十分な誤差補正シグナルを得るために近傍が多すぎず少なすぎないバランスを保っています。kが小さすぎると補正シグナルがノイズになり、大きすぎると特徴空間上で遠い無関係な患者まで含まれてしまいます。{5, 10, 20, 50}のような範囲でのkのチューニングは、今後の改善課題として明示的に挙げています。

Q16 — Why build two separate graphs?

English: This is to prevent data leakage. If validation patients were allowed to attend to each other's error signals, information from the validation set would influence the model's correction, making the evaluation optimistic. The training graph connects training patients to training patients only — used during the training phase. The validation graph connects validation patients to training patients only — validation patients can borrow error signals from training neighbours, but not from each other. This is the same principle as using a held-out test set: the model never sees information about the evaluation samples during learning.

日本語: データリークを防ぐためです。検証患者が互いの誤差シグナルを参照できると、検証セットの情報がモデルの補正に影響し、評価が楽観的になってしまいます。訓練グラフは訓練患者同士のみを接続 — 訓練フェーズで使います。検証グラフは検証患者を訓練患者のみに接続 — 検証患者は訓

練近傍の誤差シグナルは借りられますが、互いのシグナルは参照できません。これは保留テストセットを使うのと同じ原則です：モデルは学習中に評価サンプルの情報を一切見ません。

Q17 — Does memory optimization reduce accuracy?

English: No, the two optimizations do not systematically reduce accuracy. Mini-batching (256 patients per step) introduces stochasticity similar to mini-batch SGD — in fact, this mild stochasticity often acts as regularization and can improve generalization. Graph-constrained attention (attending only to $k=10$ neighbours) is not a compromise — it is the intended design of EPN. EPN should only aggregate errors from similar patients; attending to all N patients would actually introduce noise from dissimilar patients. The memory reduction is achieved by implementing the algorithm correctly, not by approximating it.

日本語： いいえ、2つの最適化は精度を体系的に低下させません。ミニバッチ処理（1ステップ256人）はミニバッチSGDと同様の確率性をもたらします — むしろこの軽度な確率性は正則化として機能し、汎化性能を向上させることがあります。グラフ制約付きアテンション（ $k=10$ の近傍のみに注目）は妥協ではなく、EPNの意図された設計です。EPNは似た患者の誤差のみを集約すべきで、 N 人全員に注目するとむしろ無関係な患者からノイズが入ります。メモリ削減はアルゴリズムを近似するのではなく、正しく実装することで達成されます。

Q18 — What is Hydra? Why use it?

English: Hydra is a configuration management framework that externalizes hyperparameters into YAML files. Instead of hardcoding values in source code, all settings — MLP hidden dimensions, learning rates, graph k , number of epochs — are declared in `run_epn.yaml`. Any parameter can be overridden from the command line without changing a single line of code. This makes experiments reproducible: two researchers running the same command with the same YAML file will get identical configurations. It also makes ablation studies easy — testing $k=20$ is just `python -m run_epn_v2 graph.k=20`.

日本語： Hydraはハイパーパラメータをソースコードの外（YAMLファイル）で管理する設定管理フレームワークです。MLPの隠れ層次元数、学習率、グラフの k 、エポック数など全ての設定が`run_epn.yaml`に宣言されます。コードを1行も変えずに、コマンドラインから任意のパラメータを上書きできます。再現性が保証されます：同じコマンドと同じYAMLファイルで実行すれば誰でも同一の設定が得られます。アブレーション実験も容易になります — $k=20$ のテストは`python -m run_epn_v2 graph.k=20`だけです。

Q19 — What are C-index and IBS? Why use both?

English: The C-index (concordance index) measures ranking accuracy: whether patients predicted to have higher risk actually fail sooner. It ranges from 0.5 (random ranking) to 1.0 (perfect), and is purely about the ordering of predictions.

The IBS (Integrated Brier Score) measures the mean squared error of survival probability predictions over time. It captures calibration — whether the predicted probabilities are numerically correct, not just correctly ordered.

We use both because they measure complementary aspects. A model can have a good C-index but poor IBS if it ranks patients correctly but gives poorly calibrated probabilities. Together they give a fuller picture of model quality.

日本語： C-index（コンコーダンス指標）はランキング精度を測ります：リスクが高いと予測された患者が実際により早く亡くなっているかどうか。0.5（ランダム）から1.0（完全）の範囲で、予測の順序だけに関わります。

IBS（Integrated Brier Score）は時間を通じた生存確率予測の平均二乗誤差を測ります。予測された生存確率が数値として正しいかどうか（キャリブレーション）を捉えます。

補完的な側面を測るため両方を使います。患者の順序は正しいが確率が不正確なモデルはC-indexは良くてIBSが悪くなります。両者を合わせることでモデル品質のより完全な評価が得られます。

Q20 — EPN uses MLP output, so isn't it essentially MLP+EPN combined?

English: Yes, and that is by design. EPN is not a replacement for the MLP — it is a correction layer on top of it. The MLP provides the baseline survival curves, and EPN refines them by aggregating error signals from similar neighbours in the patient graph. This two-stage architecture is exactly what the original `surv_epn` paper proposes. Freezing the MLP weights during Stage 2 ensures that the two components train independently, preventing mutual interference. Conceptually, EPN is “MLP plus learned neighbourhood correction.”

日本語： はい、それは意図した設計です。EPNはMLPの置き換えではなく、その上に乗る補正レイヤーです。MLPがベースラインの生存曲線を提供し、EPNは患者グラフ内の類似近傍から誤差シグナルを集約してそれを補正します。この2段階アーキテクチャはオリジナルの`surv_epn`論文が提案するもののそのものです。ステージ2でMLPの重みを凍結することで、2つのコンポーネントが独立に訓練され、相互干渉を防ぎます。概念的には、EPNは「MLP+学習された近傍補正」です。

Q21 — Why didn't MLP and EPN use the same loss function?

English: The choice of loss function is driven by how each model processes data during training. The MLP processes all training patients in a single batch, so computing the full CoxPH partial likelihood is feasible. EPN processes mini-batches of 256 patients per step. Computing the full risk set within a mini-batch that contains only a fraction of the training data gives a biased, noisy gradient. CoxCC solves this by approximating the risk set with a random control sample, which produces a statistically valid gradient even for small batches. Using the same loss for both would require either forcing the MLP into unnecessary mini-batching, or forcing EPN into impractical full-batch computation.

日本語： 損失関数の選択は、各モデルが訓練中にデータをどう処理するかによって決まります。MLPは全訓練患者を1つのバッチで処理するため、CoxPHの完全な部分尤度を計算することが可能です。

EPNは1ステップあたり256人のミニバッチで処理します。訓練データのごく一部しか含まないミニバッチ内でリスク集合全体を計算すると、偏った不安定な勾配になります。CoxCCはランダムなコントロールサンプルでリスク集合を近似することでこれを解決し、小さなバッチでも統計的に有効な勾配が得られます。両者に同じ損失関数を使うには、MLPに不必要なミニバッチ処理を強いるか、EPNに非実用的な全バッチ計算を強いるかのどちらかになります。

EXTENDED Q&A — Anticipated Questions

Q1 — Why 5-fold cross-validation?

EN: METABRIC has only 1,904 patients. With a single train/test split, the result depends heavily on which patients happen to land in the test set. 5-fold CV uses every patient for both training and validation across five rounds, giving a reliable estimate of generalisation performance with low variance. It is the standard practice for medical survival datasets of this size.

JA: METABRICは1,904名と比較的小規模なデータセットです。1回だけのtrain/test分割では、テストセットにどの患者が入るかによって結果が大きくぶれてしまいます。5-fold CVでは全患者を5回にわたってtraining/validationに使い回すため、汎化性能を安定して推定できます。この規模の医療生存分析データに対する標準的な評価手法です。

Q2 — Why these five steps? What does each step contribute?

EN: Each step addresses a specific gap between the original EPN code and a modern, reproducible pipeline.

- **Step 1 (Verify SurvivalDGM):** Establish the existing baseline performance so later comparisons are fair.
- **Step 2 (Wrap MLP as Lightning module):** Replace ad-hoc training loops with a standardised framework, enabling checkpointing, logging, and GPU switching out of the box.
- **Step 3 (Build patient similarity graph with PyG):** Enable the EPN's neighbourhood-based error correction; without the graph, EPN cannot run.
- **Step 4 (Reimplement EPN as tensor-only Lightning module):** Remove the DataFrame dependency from the original code so the model integrates cleanly with the Lightning data pipeline.
- **Step 5 (Add Hydra):** Make every hyperparameter configurable from the command line without touching source code, enabling reproducible and comparable experiments.

JA: 各ステップは、元のEPNコードとモダンで再現性のあるパイプラインとの間にある具体的なギャップを埋めるためのものです。

- **Step 1 (SurvivalDGMの確認) :** 後の比較が公平になるよう、既存のベースライン性能を確立する。

- **Step 2 (MLPをLightningモジュールとしてラップ)** : 場当たりのtraining loopを標準化されたフレームワークに置き換え、checkpointingやlogging、GPU切り替えをすぐに使えるようにする。
- **Step 3 (PyGを用いた患者類似性グラフの構築)** : EPNの近傍ベースの誤差修正を実現する。グラフなしではEPNは機能しない。
- **Step 4 (EPNをtensor-onlyのLightningモジュールとして再実装)** : 元コードのDataFrame依存を取り除き、LightningのデータパイプラインとクリーンにIntegrateできるようにする。
- **Step 5 (Hydraの追加)** : ソースコードを変更せずにコマンドラインから全ハイパーパラメータを設定可能にし、実験の再現性と比較可能性を確保する。

Q3 — Why compare SurvivalDGM, MLP, and EPN? What does each model represent?

EN: The three models serve distinct roles in the evaluation story.

- **SurvivalDGM** is the existing baseline from our supervisor. It uses a probabilistic graph and achieves C-index 0.6043. We need this as a reference point to show whether our work improves over the prior state of the art in this pipeline.
- **MLP** is our Stage 1 model — a simple deterministic feedforward network. Comparing it against SurvivalDGM shows that even without a graph, a well-tuned MLP (C-index 0.6495) already outperforms the probabilistic baseline.
- **EPN** is our main contribution — it adds graph-based error correction on top of the MLP. Comparing EPN against both models isolates the contribution of the graph correction step.

Without all three comparisons, we cannot answer the key question: does adding a graph actually help?

JA: 3つのモデルはそれぞれ異なる役割を持ちます。

- **SurvivalDGM** は指導教員 (Valentin) による既存のベースラインです。確率的グラフを使用し、C-index 0.6043を達成します。このパイプラインにおける先行モデルからの改善を示すための基準点として必要です。
- **MLP** は私たちのStage 1モデルで、シンプルな決定論的フィードフォワードネットワークです。SurvivalDGMとの比較によって、グラフなしでも適切に調整されたMLP (C-index 0.6495) が確率的ベースラインを上回ることが示されます。
- **EPN** は私たちのメイン貢献であり、MLPにグラフベースの誤差修正を追加したものです。EPNと両モデルの比較によって、グラフ修正ステップの貢献を切り分けることができます。

この3つの比較がなければ「グラフを追加することで実際に精度が向上するか」という核心的な問いに答えられません。

Q4 — Is the implementation MLP+EPN integrated into the pipeline? What happened to SurvivalDGM?

EN: Yes, exactly. We integrated the MLP+EPN two-stage pipeline as new code within Valentin's existing repository. SurvivalDGM was not removed or modified — it remains in the repository and is still usable. We evaluate it as a comparison baseline using `evaluate_dgm.py`. Our additions sit alongside it as a separate, parallel pipeline. The repository now contains two complete pipelines: the original SurvivalDGM and the new MLP->EPN pipeline.

JA: はい、その通りです。ValentinのリポジトリにMLP+EPNの二段階パイプラインを新たなコードとして追加・統合しました。SurvivalDGMは削除も変更もしておらず、リポジトリにそのまま残っています。evaluate_dgm.pyでベースライン比較として評価するために使用します。私たちの追加分は既存のSurvivalDGMと並存する別のパイプラインとして共存しています。リポジトリには現在、元のSurvivalDGMと新しいMLP->EPNパイプラインの2つの完全なパイプラインが含まれています。

Q5 — What are the 9 input features to the MLP?

EN: The METABRIC dataset in pycox provides 9 clinical features for each patient. Continuous features (x0, x1, x2, x3, x8) are standardised with StandardScaler; binary/ordinal features (x4-x7) are used as-is.

- x0: Age at diagnosis (continuous)
- x1: Tumor size in mm (continuous)
- x2: Number of lymph nodes examined positive (continuous)
- x3: Nottingham Prognostic Index score (continuous)
- x4: ER positive status (binary 0/1)
- x5: Histological grade 1/2/3 (ordinal)
- x6: PR positive status (binary 0/1)
- x7: HER2 positive status (binary 0/1)
- x8: ER expression level, continuous measurement (continuous)

JA: pycoxが提供するMETABRICデータセットには1患者あたり9つの臨床特徴量があります。連続値特徴量 (x0, x1, x2, x3, x8) はStandardScalerで標準化し、二値・順序値特徴量 (x4-x7) はそのまま使用します。

- x0: 診断時年齢 (連続値)
- x1: 腫瘍サイズ (mm、連続値)
- x2: 陽性リンパ節数 (連続値)
- x3: Nottingham予後指数スコア (連続値)
- x4: ER陽性ステータス (二値 0/1)
- x5: 組織学的グレード1/2/3 (順序値)
- x6: PR陽性ステータス (二値 0/1)
- x7: HER2陽性ステータス (二値 0/1)
- x8: ER発現量、連続値計測 (連続値)

Q6 — Why does the MLP use CoxPH but EPN uses CoxCC? What do they mean?

EN: CoxPH (Cox Partial Hazard loss): At each event time, the full risk set — all patients who have not yet experienced the event — is used to compute the log-partial likelihood. This is the exact formulation, statistically rigorous, and appropriate when the dataset is small enough for the full risk set to fit in memory. We use it for the MLP because Stage 1 trains on the full training set in one pass.

CoxCC (Cox Case-Control loss): Instead of the full risk set, a small random sample of controls is drawn for each case. This is a well-established approximation that produces unbiased gradient estimates at a fraction of the computational cost. pycox recommends CoxCC as the default for deep survival models.

The EPN was originally designed with CoxCC, and we preserved that design. The two-stage architecture also means EPN processes patients in mini-batches of 256, where sampling controls per step is more natural. That said, using different loss functions is acknowledged as a limitation in our discussion slide.

JA: CoxPH（Cox部分ハザード損失）：各イベント時刻において、まだイベントを経験していない全患者（リスクセット）を使って対数部分尤度を計算します。これは統計的に厳密な定式化であり、データセットが小さく全リスクセットがメモリに収まる場合に適しています。Stage 1のMLPは全学習データを1パスで学習するため、この損失を使用しています。

CoxCC（Cox Case-Control損失）：全リスクセットの代わりに、各ケースに対して少数のコントロールをランダムサンプリングします。これは確立された近似であり、計算コストを大幅に削減しながら不偏な勾配推定を実現します。pycoxはこれを深層生存分析モデルのデフォルトとして推奨しています。

EPNはCoxCCを用いて設計されており、私たちはその設計を踏襲しました。二段階アーキテクチャではEPNが256患者のミニバッチで処理するため、コントロールのサンプリングが自然でもあります。ただし異なる損失関数の使用はディスカッションスライドで制限として認めています。

Q7 — Which file do you actually run for experiments? Is creating run_epn_v2.py the true goal of this project?

EN: To run the full MLP->EPN pipeline across all 5 folds:

```
cd "Vanlentin's pipeline" python -m survival_analysis.experiments.run_epn_v2
```

This single command runs Stage 1 (MLP) and Stage 2 (EPN) for all five splits and saves results to results_comparison_v2.csv.

Regarding the project goal: run_epn_v2.py is the entry point, but it is not the goal in itself. The true goal is a complete, clean, reproducible integration of EPN into the Lightning pipeline. The core contributions are the tensor-only EPN reimplementation (epn_module.py), the two-graph data leakage prevention (graph_builder.py, epn_datamodule.py), and the memory optimisation. run_epn_v2.py is the orchestration that ties them together.

JA: 5-fold全体のMLP->EPNパイプラインを実行するには：

```
cd "Vanlentin's pipeline" python -m survival_analysis.experiments.run_epn_v2
```

このコマンド1つでStage 1 (MLP) とStage 2 (EPN) が5分割全て実行され、結果が `results_comparison_v2.csv` に保存されます。

プロジェクトの目的について：`run_epn_v2.py`はエントリーポイントですが、それ自体が目的ではありません。本当の目的はEPNをLightningパイプラインに完全・クリーン・再現可能な形で統合することです。中核的な貢献はtensor-onlyのEPN再実装 (`epn_module.py`)、データリークage防止のための2グラフ設計 (`graph_builder.py`、`epn_datamodule.py`)、そしてメモリ最適化です。`run_epn_v2.py`はそれらを束ねるオーケストレーションです。

Q8 — What is the role of each new file? Which files are truly essential?

EN: Essential — the pipeline cannot run without these: - `mlp_module.py`: Stage 1 MLP as a Lightning module - `epn_module.py`: Stage 2 EPN as a tensor-only Lightning module - `graph_builder.py`: builds the patient k-NN similarity graphs - `epn_datamodule.py`: precomputes MLP predictions, builds graphs, serves batches to EPN - `metabric_datamodule.py`: loads and splits the METABRIC data - `run_epn_v2.py`: the entry point that runs the full pipeline - `configs/experiment/run_epn.yaml`: Hydra config holding all hyperparameters

Useful for evaluation/comparison but not strictly required to train: - `evaluate_dgm.py`: evaluates SurvivalDGM for the baseline comparison - `run_all_splits.py`: runs SurvivalDGM across all 5 splits - `run_mlp.py`: trains and evaluates the MLP alone (useful for debugging Stage 1)

Optional / Supplementary: - `visualize.py`: generates survival curve plots and patient graph visualisation - `run_epn.py`: an older intermediate version, superseded by v2

JA: 必須 — これらなしではパイプラインが動作しない： - `mlp_module.py`: Stage 1 MLPをLightningモジュールとして実装 - `epn_module.py`: Stage 2 EPNをtensor-onlyのLightningモジュールとして実装 - `graph_builder.py`: 患者k-NN類似性グラフを構築 - `epn_datamodule.py`: MLP予測の事前計算、グラフ構築、EPNへのバッチ提供 - `metabric_datamodule.py`: METABRICデータの読み込みと分割 - `run_epn_v2.py`: パイプライン全体を実行するエントリーポイント - `configs/experiment/run_epn.yaml`: 全ハイパーパラメータを保持するHydra設定

評価・比較に有用だが学習には不要： - `evaluate_dgm.py`: SurvivalDGMをベースライン比較として評価 - `run_all_splits.py`: 比較用にSurvivalDGMを5分割全体で実行 - `run_mlp.py`: Stage 1のデバッグに役立つMLP単体の学習・評価

オプション・補助的： - `visualize.py`: 生存曲線プロットと患者グラフ可視化 - `run_epn.py`: 古い中間バージョン、v2に置き換え済み

Q9 — Why 110 epochs for MLP and 50 epochs for EPN?

EN: These values were determined empirically by monitoring validation loss convergence.

MLP (110 epochs): The CoxPH loss on the METABRIC training set typically converges around 100-120 epochs with Adam at $\text{lr}=0.001$. We set 110 as the value where validation loss stabilises without significant overfitting.

EPN (50 epochs): EPN starts from a frozen, already-trained MLP and only learns the error correction on top. The correction signal is relatively small compared to the base prediction, so the model converges faster. 50 epochs was found sufficient for the correction weights to stabilise, and more epochs showed no meaningful improvement.

Both values are set in the Hydra config and can be overridden from the command line for future experiments.

JA: これらの値はvalidation lossの収束を観察することで経験的に決定されました。

MLP (110エポック) : METABRICの学習データに対するCoxPH lossは、Adam (lr=0.001) で通常100~120エポック前後で収束します。110はvalidation lossが大きな過学習なしに安定する値として設定しました。

EPN (50エポック) : EPNはフリーズされた学習済みMLPから始まり、その上で誤差修正のみを学習します。修正シグナルはベース予測に比べて比較的小さいため、モデルはより速く収束します。50エポックで修正の重みが安定し、より多くのエポックでも意味のある改善は見られませんでした。

両方の値はHydra設定に書かれており、将来の実験ではコマンドラインから上書き可能です。

Q10 — How do the files interact during two-stage training? Which file actually runs the training?

EN: Training is orchestrated by run_epn_v2.py. Here is the flow:

Stage 1: 1. run_epn_v2.py creates MetabricGraphSurvivalDataModule (reads METABRIC, applies the split). 2. It instantiates MLPModule and calls pl.Trainer.fit(mlp, datamodule=base_dm). 3. Lightning calls base_dm.train_dataloader() each epoch, returning training graph data. 4. After training, mlp.freeze() locks all MLP weights.

Stage 2: 5. run_epn_v2.py creates EPNDataModule(base_datamodule=base_dm, mlp_module=mlp). 6. EPNDataModule.setup() calls the frozen MLP to precompute survival curves, then calls graph_builder.py to build the two k-NN graphs. 7. run_epn_v2.py instantiates EPNModule and calls pl.Trainer.fit(epn, datamodule=epn_dm). 8. Lightning calls epn_dm.train_dataloader() each epoch, sampling mini-batches of 256 patients. 9. EPNModule.training_step() computes CoxCC loss and backpropagates through EPN weights only.

JA: 学習はrun_epn_v2.pyによってオーケストレーションされます。フローは以下の通りです：

Stage 1: 1. run_epn_v2.pyがMetabricGraphSurvivalDataModuleを作成（METABRICを読み込み、分割を適用）。2. MLPModuleをインスタンス化し、pl.Trainer.fit(mlp, datamodule=base_dm)を呼び出す。3. Lightningはエポックごとにbase_dm.train_dataloader()を呼び出し、学習グラフデータを返す。4. 学習後、mlp.freeze()で全MLPの重みをロック。

Stage 2: 5. run_epn_v2.pyがEPNDataModule(base_datamodule=base_dm, mlp_module=mlp)を作成。6. EPNDataModule.setup()がフリーズ済みMLPを呼び出して生存曲線を事前計算し、graph_builder.pyを呼んで2つのk-NNグラフを構築。7. run_epn_v2.pyがEPNModuleをインスタンス化し、pl.Trainer.fit(epn, datamodule=epn_dm)を呼び出す。8. Lightningはエポックごとにepn_dm.train_dataloader()を呼び出し、256患者のミニバッチをサンプリング。9.

EPNModule.training_step()がCoxCC lossを計算し、EPNの重みのみを通じてバックプロパゲーション。

Q11 — Why PyTorch Lightning? What is it?

EN: PyTorch Lightning is a high-level framework built on top of standard PyTorch. It separates the “what” (the model and loss) from the “how” (the training loop boilerplate). You define training_step(), validation_step(), and configure_optimizers() in your module; Lightning handles the epoch loop, device placement, gradient zeroing, and optimizer stepping automatically.

We chose it because: (1) it enforces the clean model/data separation our supervisor required; (2) switching between CPU and GPU is one config line; (3) built-in checkpointing and logging with minimal code; (4) it is the industry standard for research-grade PyTorch, making the project easier to maintain and hand off.

JA: PyTorch Lightningは標準PyTorchの上に構築された高レベルフレームワークです。「何を」（モデルと損失）と「どのように」（training loopのボイラープレート）を分離します。training_step()、validation_step()、configure_optimizers()をモジュール内で定義すれば、Lightningがエポックループ、デバイス配置、勾配のゼロ化、オプティマイザのステップを自動的に処理します。

採用理由：(1) 指導教員が求めたクリーンなモデル/データの分離を強制する；(2) CPUとGPUの切り替えが設定1行；(3) 最小限のコードで組み込みのcheckpointingとlogging；(4) 研究グレードのPyTorchコードの業界標準で引き継ぎと保守が容易。

Q12 — When is the patient similarity graph used in two-stage training? Why was it created?

EN: The graph is built during the setup phase between Stage 1 and Stage 2, inside EPNDataModule.setup(). It is not used during MLP training at all.

During Stage 2 EPN training, the graph is used in every forward pass: for each patient in the mini-batch, the graph provides its k=10 nearest neighbours' indices. EPN looks up those neighbours' MLP prediction errors and computes a weighted correction via attention.

The graph was created because EPN's core hypothesis is that patients similar in feature space tend to make similar MLP errors. By connecting similar patients, EPN can leverage the neighbourhood error signal to correct its own prediction. Without the graph, there is no way to identify which patients are “similar” at inference time.

JA: グラフはStage 1とStage 2の間のsetupフェーズ、具体的にはEPNDataModule.setup()内で構築されます。MLPの学習中には一切使用されません。

Stage 2のEPN学習中、グラフは全てのforward passで使用されます：ミニバッチ内の各患者に対して、グラフがk=10の最近傍のインデックスを提供します。EPNはそれらの近傍患者のMLP予測誤差を参照し、注意機構を通じて重み付き修正を計算します。

グラフを作成した理由は、EPNのコア仮説が「特徴空間で類似した患者はMLP予測誤差も類似する傾向がある」というものだからです。類似患者を接続するグラフにより、EPNは近傍の誤差シグナルを活用して自身の予測を修正できます。グラフなしでは推論時に「どの患者が類似しているか」を特定できません。

Q13 — Why use PyTorch Geometric?

EN: PyTorch Geometric (PyG) is the standard library for graph neural networks in PyTorch. We use it specifically for its `knn_graph` utility, which efficiently computes k-nearest-neighbour graphs from a feature matrix using GPU-accelerated distance computation. Writing a correct and efficient k-NN graph from scratch would require significant engineering. PyG also provides the `Data` object that natively stores node features and edge indices as tensors, integrating cleanly with our tensor-only design. It is the natural choice for any graph-based learning work in PyTorch.

JA: PyTorch Geometric (PyG) はPyTorchにおけるグラフニューラルネットワークの標準ライブラリです。特にその`knn_graph`ユーティリティを使用しており、特徴行列からGPU加速の距離計算を用いて効率的にk-NNグラフを計算します。正確で効率的なk-NNグラフをゼロから書くには相当なエンジニアリングが必要です。PyGはノード特徴量とエッジインデックスをテンソルとしてネイティブに格納するDataオブジェクトも提供しており、tensor-only設計とクリーンに統合できます。PyTorchにおけるグラフベース学習には自然な選択肢です。

Q14 — What is cosine similarity? Why use it here?

EN: Cosine similarity measures the angle between two vectors, ignoring their magnitudes. Two patients with similar feature profiles — regardless of scale — will have a high cosine similarity close to 1.0.

We chose it because the METABRIC features are heterogeneous: some are continuous and standardised (age, tumor size), while others are binary flags (ER, HER2 status). Euclidean distance would be dominated by continuous features with large numeric ranges. Cosine similarity is scale-invariant and therefore treats all dimensions more equally. It is also the default metric in PyG's `knn_graph` and is well-established for patient similarity in clinical data.

JA: コサイン類似度は2つのベクトル間の角度を測定し、大きさは無視します。スケールに関係なく類似した特徴プロファイルを持つ2人の患者は、1.0に近い高いコサイン類似度を持ちます。

コサイン類似度を選んだ理由はMETABRICの特徴量が異種混合だからです：連続値で標準化されたもの（年齢、腫瘍サイズ）と二値フラグ（ER・HER2ステータス）が混在しています。ユークリッド距離では数値範囲が大きい連続特徴量に支配されてしまいます。コサイン類似度はスケール不変であるため全次元をより均等に扱います。またPyGの`knn_graph`のデフォルト指標でもあり、臨床データでの患者類似性に広く確立されています。

Q15 — Why k=10 for the patient similarity graph?

EN: $k=10$ balances neighbourhood richness against noise. Too small ($k=3$) means the error correction signal is weak and unstable. Too large ($k=50$) means distant, less-similar patients are included, introducing noise. $k=10$ is a commonly used default in graph-based clinical learning literature and was the value used in the original EPN paper. Tuning k over $\{5, 10, 20, 50\}$ is also listed as a future improvement direction.

JA: $k=10$ は近傍の豊富さとノイズのバランスを取る値です。小さすぎる場合 ($k=3$)、誤差修正シグナルが弱く不安定になります。大きすぎる場合 ($k=50$)、類似度の低い遠い患者も含まれノイズが混入します。 $k=10$ はグラフベースの臨床学習文献でよく使われるデフォルト値であり、元のEPN論文でも使用されていた値です。 $\{5, 10, 20, 50\}$ で k を調整することは将来の改善方向の一つとしても列挙しています。

Q16 — Why build two separate graphs?

EN: This prevents data leakage. EPN computes error corrections by looking at neighbours' MLP prediction errors. If validation patients could attend to other validation patients as neighbours, the model would effectively use information from the validation set during inference, artificially inflating validation metrics.

Training graph: training patients to training patients only. Validation graph: validation patients look up training patients only — they cannot see each other's errors.

This ensures the validation evaluation is honest and measures true generalisation.

JA: これはデータリークage防止のための措置です。EPNは近傍患者のMLP予測誤差を見て修正を計算します。もし検証患者が他の検証患者を近傍として参照できると、推論中に検証セットの情報を事実上使用することになり、検証指標が人工的に高くなります。

学習グラフ：学習患者同士のみを接続。検証グラフ：検証患者は学習患者のみを近傍として参照 — 検証患者同士はお互いの誤差を見ることができない。

これにより検証評価が正直になり、真の汎化性能を測定できます。

Q17 — Does the memory optimisation reduce accuracy?

EN: No — neither optimisation reduces accuracy.

Mini-batching (256 patients per step): This is standard stochastic gradient descent. Each epoch still covers the full training set on average. Mini-batching can act as a regulariser, sometimes improving generalisation.

Graph-constrained attention ($k=10$ neighbours): Restricting attention to the k nearest neighbours is actually a feature, not a limitation. Attending to all patients would include very dissimilar patients, introducing noise into the correction. Focusing on the 10 most similar patients concentrates the correction on the most relevant signal.

JA: いいえ — どちらの最適化も精度を低下させません。

ミニバッチ（1ステップ256患者）：これは標準的な確率的勾配降下法です。各エポックでは平均して全学習セットをカバーします。ミニバッチは正則化として機能し、汎化性能を改善することすらあります。

グラフ制約付き注意（ $k=10$ 近傍）：注意を k 個の最近傍に制限することは制限ではなく特徴です。全患者を対象にすると非常に非類似な患者も含まれ、修正にノイズが混入します。最も類似した10患者に集中することで、最も関連性の高いシグナルに絞った修正が可能になります。

Q18 — What is Hydra and why use it?

EN: Hydra is a configuration management framework by Meta AI. Instead of hardcoding hyperparameters in Python files, Hydra reads structured YAML configs and exposes every setting as an override-able command-line argument.

Example — run with a larger graph and more epochs, no code change needed: `python -m survival_analysis.experiments.run_epn_v2 graph.k=20 epn.epochs=100`

We use Hydra because it makes experiments reproducible (config is stored alongside output), comparable (every run has a logged config), and easy to sweep (built-in multirun support for grid searches). It is far better than editing Python files directly for research requiring many hyperparameter combinations.

JA: HydraはMeta AIが開発した設定管理フレームワークです。PythonファイルにハードコードされたハイパーパラメータをYAML設定ファイルに分離し、全設定をコマンドラインから上書き可能な引数として公開します。

例 — コードを変更せずに大きなグラフとより多いエポックで実行： `python -m survival_analysis.experiments.run_epn_v2 graph.k=20 epn.epochs=100`

Hydraを使う理由：実験を再現可能（設定が出力と一緒に保存）、比較可能（全実行にログ付き設定）、スイープが容易（グリッドサーチ用の組み込みマルチラン機能）にするためです。多くのハイパーパラメータの組み合わせを試す研究にはPythonファイルの直接編集よりはるかに優れています。

Q19 — What do C-index and IBS measure? Why use both?

EN: C-index (Concordance Index): A discrimination metric. It measures whether patients who die earlier receive higher predicted risk scores — how well the model ranks patients by survival time. 0.5 = random; 1.0 = perfect. It does not measure how accurate the survival curve probability values are, only the ranking.

IBS (Integrated Brier Score): A calibration metric. At each time point, the Brier Score is the mean squared error between predicted survival probability and the observed outcome. IBS integrates this over the full time horizon. Lower is better. It measures whether the predicted probability values themselves are accurate.

We use both because they are complementary: a model could rank patients correctly (good C-index) but have poorly calibrated probabilities (bad IBS), or vice versa. Together they give a

complete picture of model quality.

JA: C-index（コンコーダンス指数）：識別能力の指標。早期に死亡する患者が長期生存患者より高いリスクスコアを得るか — モデルが患者を生存時間でどれだけ正確にランク付けできるかを測定します。0.5=ランダム、1.0=完全。生存確率値の正確さではなく、ランキングのみを測定します。

IBS（積分ブライアースコア）：較正精度の指標。各時点でブライアースコアは予測生存確率と観測結果の平均二乗誤差です。IBSはこれを時間軸全体で積分したもので、低いほど良い。実際の生存確率値がどれだけ正確かを測定します。

両方を使う理由はそれらが補完的だからです：患者を正確にランク付けできても（良いC-index）確率値の較正が悪い（悪いIBS）場合もあります。合わせてモデル品質の完全な全体像を提供します。

Q20 — EPN takes MLP output as input, so isn't it effectively MLP + EPN?

EN: Yes, exactly. EPN is not a standalone model — it is a correction layer on top of the MLP. The final prediction is:

EPN output = MLP survival curve + graph-based error correction

What we evaluate as “EPN” is in practice the combined MLP+EPN system. The MLP's weights are frozen during Stage 2, so EPN learns only the additive correction. The comparison between MLP alone (C-index 0.6495) and MLP+EPN (C-index 0.6157) directly shows whether the graph correction step adds value.

JA: はい、まさにその通りです。EPNはスタンドアロンモデルではなく、MLPの上に乗る修正レイヤーです。最終予測は：

EPN出力 = MLP生存曲線 + グラフベースの誤差修正

「EPN」として評価しているものは実際にはMLP+EPNの複合システムです。Stage 2ではMLPの重みがフリーズされているため、EPNは加算的な修正のみを学習します。MLP単体（C-index 0.6495）とMLP+EPN（C-index 0.6157）の比較により、グラフ修正ステップが価値を加えるかどうかが直接示されます。

Q21 — “MLP and EPN use different loss functions.” Then why didn't you use the same loss function?

EN: There are two practical reasons.

First, the original EPN design used CoxCC. The `surv_epn` codebase trains EPN with CoxCC. Reimplementing faithfully meant preserving the loss to keep results comparable to the original paper.

Second, CoxPH is incompatible with mini-batch training. CoxPH requires the full risk set at each event time. With mini-batches of 256 patients, the exact risk set cannot be computed — you would need all 1,523 patients in memory every step, defeating the purpose of mini-batching. CoxCC samples a small set of controls per case by design, making it naturally compatible with mini-batch training.

Switching EPN to CoxPH is a valid future improvement, explicitly listed on our Future Improvements slide. It would enable a fairer ablation study isolating the graph's contribution.

JA: 実際的な理由が2つあります。

第一に、元のEPN設計はCoxCCを使用していました。surv_epnコードベースはCoxCCでEPNを学習します。EPNを忠実に再実装するためには、元の論文との比較可能性を保つために損失関数を維持する必要がありました。

第二に、CoxPHはミニバッチ学習と互換性がありません。CoxPHは各イベント時刻でフルリスクセットを必要とします。256患者のミニバッチでは正確なリスクセットを計算できません — 毎ステップで全1,523患者をメモリに入れる必要があり、ミニバッチの目的を果たせなくなります。CoxCCは設計上少数のコントロールをサンプリングするため、ミニバッチ学習と自然に互換性があります。

EPNをCoxPHに変更することは有効な将来の改善であり、「Future Improvements」スライドに明示的に記載しています。これによりグラフの貢献をより公平に切り分けるアブレーション研究が可能になります。